

An Energy Efficient Layered Decoding Architecture for LDPC Decoder

Jie Jin and Chi-ying Tsui, *Member, IEEE*

Abstract—Low-density parity-check (LDPC) decoder requires large amount of memory access which leads to high energy consumption. To reduce the energy consumption of the LDPC decoder, memory-bypassing scheme has been proposed for the layered decoding architecture which reduces the amount of access to the memory storing the soft posterior reliability values. In this work, we present a scheme that achieves the optimal reduction of memory access for the memory bypassing scheme. The amount of achievable memory bypassing depends on the decoding order of the layers. We formulate the problem of finding the optimal decoding order and propose algorithm to obtain the optimal solution. We also present the corresponding architecture which combines some of memory components and results in reduction of memory area. The proposed decoder was implemented in TSMC 0.18 μm CMOS process. Experimental results show that for a LDPC decoder targeting IEEE 802.11n specification, the amount of memory access values can be reduced by 12.9–19.3% compared with the state-of-the-art design. At the same time, 95.6%–100% hardware utilization rate is achieved.

Index Terms—Low power, low-density parity-check code, simulated annealing.

I. INTRODUCTION

RECENTLY, low-density parity-check (LDPC) codes [1] have gained significant attention due to their near Shannon limit performance [2]. They have been adopted in several wireless standards, such as DVB-S2 [3], IEEE 802.16e [4] and IEEE 802.11n [5], because of their excellent error correcting performance.

A LDPC code is a linear block code defined by a sparse parity check matrix [Fig. 1(a)]. It can be represented by a bipartite graph, called Tanner Graph [6] as shown in Fig. 1(b), which contains two sets of nodes: variable nodes that represent the bits of a codeword and check nodes that implement the parity-check constraints. The standard decoding procedure is the message passing algorithm, also known as “sum-product” or “belief propagation” (BP) algorithm [2], which iteratively exchanges the messages between the check nodes and the variable nodes along the edges of the graph. In the original message passing algorithm, the messages first are broadcasted to all the check nodes from the variable nodes and then along the edges of the graph the updated messages are fed back from the check nodes to the variable nodes to finish one iteration of decoding. There

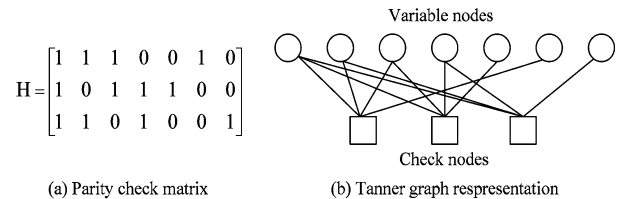


Fig. 1. Example of parity check matrix of a LDPC code and its Tanner graph representation.

are different ways to implement the LDPC decoder, based on the number of processing units available. In general, the LDPC decoder architecture can be classified into three types: fully parallel architecture, serial architecture and partial parallel architecture. In fully parallel architecture [7], a check node processor is needed for every check node, which usually results in large hardware cost and complicated routing, and hence is less flexible. The serial architecture uses just one check node processor to share the computation of all the check nodes and is too slow for most applications. For partial parallel architectures, multiple processing units are used allowing proper tradeoff between the hardware cost and the throughput and are commonly adopted in the actual implementation [9]–[23]. In order to achieve higher convergence speed, i.e., to minimize the number of decoding iteration, serial message passing algorithm, also known as layered decoding algorithm, has been proposed [8], [9] together with the corresponding partial parallel architecture. There are two types of layered decoding schemes: vertical layered decoding and horizontal layered decoding [8]. In the horizontal layered decoding, a single or a certain number of check nodes (called layer) are first updated. Then the whole set of neighboring variable nodes are updated, and the decoding process proceeds layer after layer. Dually, in the vertical layered decoding, a single or a certain number of variable nodes (layer of variable nodes) may be updated first. Then the whole set of neighboring check nodes are updated [11]. Because the serial check node processor is easier to be implemented in VLSI and therefore the horizontal layered decoding is preferable for practical implementations [11]. Because of the faster convergence and regular architecture, layered decoders are commonly found in the LDPC decoder implementation [9]–[18]. In this work, we focus on the LDPC decoding implementation based on the layered decoding algorithm.

High power consumption is one of the bottlenecks for LDPC decoder design. Recently low power design techniques and architectures have been proposed for fully parallel [26], [27], and partial-parallel [10] LDPC decoder architecture. The partial parallel architectures based on the layered decoding algorithm have efficiently reduced the hardware cost and sped up the convergence rate, and already led to energy-efficient design

Manuscript received September 15, 2008; revised February 08, 2009; accepted March 16, 2009. First published September 01, 2009; current version published July 23, 2010. This work was supported in part by Hong Kong RGC CERF under Grant 620408.

The authors are with the Department of Electronic and Computer Engineering, the Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Hong Kong (e-mail: eejinjie@ust.hk).

Digital Object Identifier 10.1109/TVLSI.2009.2021479

comparing with other architectures. However how to further reduce its energy consumption is still a challenging design problem. Due to the large amount of memory access, the power consumption of the memory access accounts for the major part of the total power consumption of the layer decoder [9]. Reducing the energy consumption of the memories is the key issue to realize a low energy LDPC decoder. At the algorithmic level, the Min-sum decoding algorithm [24] and its variants have been proposed, which greatly reduces the memory storage required for the check to variable messages [23], and also the energy consumption of the memories of the LDPC decoder with insignificant performance loss. At the architectural level, memory-bypassing scheme has been proposed in [10] to reduce the amount of the memory access by utilizing the characteristic of the LDPC parity check matrix and the decoding algorithm for the layered decoding architecture. However, the scheme proposed in [10] may not lead to optimal reduction of memory access. In this work, we propose several schemes that lead to the optimal reduction of the memory access. Compared with the previous works, this work has the following contributions. Firstly by de-coupling the read and write access order of the memory storing the soft posterior reliability values (we denote it as the Channel RAM), optimal amount of memory bypassing is achieved and the number of idle clock cycles is reduced. Secondly, we are not only considering memory bypassing between consecutive layers of decoding but extending it to multiple layers. By doing so, the number of memory bypass is maximized. Thirdly, the decoding order of the layers has significant impact on the amount of memory bypass that can be achieved. We formulate the problem of finding the optimal decoding order given a parity check matrix as a searching problem and propose algorithm to obtain the optimal decoding order. Fourthly, based on the decoupling of the read and write order of the Channel RAM, we propose a memory efficient architecture, in which the Channel RAM and the memory storing the intermediate data are merged into a single memory. By doing so, the overall area is reduced.

The rest of the paper is organized as follows. Section II gives the background of LDPC decoding scheme, the traditional layered LDPC decoder architecture and the memory-bypassing scheme for the layered decoding architecture proposed in [10]. The proposed memory bypassing scheme is presented in Section III. A quick searching algorithm to find the optimal decoding order of the layers of the LDPC base matrix, which results in the maximum overlapping, is described in Section IV. The overall decoder architecture implementing the memory-bypassing scheme is described in Section V. In Section VI, experimental results and comparisons among different LDPC decoders are presented. Conclusions are drawn in Section VII.

II. BACKGROUND

A. LDPC Decoding Algorithms

LDPC codes are linear block codes characterized by a sparse matrix, called parity-check matrix. The set of valid codewords C is defined as

$$H \bullet x^T = 0 \quad \forall x \in C. \quad (1)$$

The code can also be described by means of a bipartite graph, known as Tanner graph (Fig. 1(b)) [6]. A Tanner graph is made up of two entities, variable nodes (VN) and check nodes (CN), and they are connected to each other through a set of edges. An edge links the check node m to the variable node n if the element $H_{m,n}$ of the parity check matrix is non-null. The optimal LDPC decoding is achieved by using a message passing algorithm, also known as “belief propagation” (BP) [2], which can be described as an iterative exchange of messages along the edges of the Tanner graph [6]. The algorithm proceeds iteratively until a maximum number of iterations are elapsed or a stopping rule is met. Inputs of the algorithm are the intrinsic Log-Likelihood Ratios (LLRs) of the received bits (i.e., the variable nodes), also referred to as *a priori* information.

The following describes the belief propagation algorithm. $R_{m,n}^{(q)}$ is the check-to-variable message from check node m to variable node n in the q th iteration. $Q_{m,n}^{(q)}$ is the variable-to-check message from variable node n to check node m in the q th iteration. M_n is the set of the neighboring check nodes of variable node n , and N_m is the set of the neighboring variable nodes of check node m . In the q th iteration, the variable node process and the check node process are computed as follows.

Variable node process: the variable node n receives the messages $R_{m,n}^{(q)}$ from the neighboring check nodes and propagates back the updated messages $Q_{m,n}^{(q)}$ as

$$Q_{m,n}^{(q)} = \lambda_n + \sum_{i \in \{M_n \setminus m\}} R_{i,n}^{(q)} \quad (2)$$

where λ_n is the intrinsic LLR of the variable node n . At the same time, the posterior reliability value, also referred to as the soft output for the variable node n , is given by

$$\Lambda_n^{(q)} = \lambda_n + \sum_{i \in \{M_n\}} R_{i,n}^{(q)}. \quad (3)$$

Check node process: the check node m combines together messages $Q_{m,n}^{(q)}$ from the neighboring variable nodes to compute the updated messages $R_{m,n}^{(q+1)}$, which are sent back to the corresponding variable nodes. Updates are performed separately on signs and magnitudes as follows:

$$-\text{sgn}(R_{m,n}^{(q+1)}) = \prod_{j \in \{N_m \setminus n\}} -\text{sgn}(Q_{m,j}^{(q)}) \quad (4)$$

$$\left| R_{m,n}^{(q+1)} \right| = \Phi^{-1} \left\{ \sum_{j \in \{N_m \setminus n\}} \Phi \left(\left| Q_{m,j}^{(q)} \right| \right) \right\} \quad (5)$$

where

$$\Phi(x) = \Phi^{-1}(x) = -\ln \left(\tanh \left(\frac{x}{2} \right) \right). \quad (6)$$

The layered decoding scheduling improves the convergence speed and reduces the number of iteration by viewing the parity check as a sequence of check through horizontal or vertical layers. The intermediate updated messages are used in the updating of the next layer. The layered decoding principle for horizontal layers is expressed by [14]

$$-\text{sgn}\left(R_{m,n}^{(q+1)}\right) = \prod_{j \in \{N_m \setminus n\}} -\text{sgn}\left(\Gamma_{m,j}^{(q+1)}\right) \quad (7)$$

$$\left|R_{m,n}^{(q+1)}\right| = \Phi^{-1}\left\{\sum_{j \in \{N_m \setminus n\}} \Phi\left(\left|\Gamma_{m,j}^{(q+1)}\right|\right)\right\} \quad (8)$$

and

$$\Gamma_{m,n}^{(q+1)} = \Lambda_n^{(q+1)}[k-1] - R_{m,n}^{(q)} \quad (9)$$

$$\Lambda_n^{(q+1)}[k] = \Gamma_{m,n}^{(q+1)} + R_{m,n}^{(q+1)} \quad (10)$$

where $k \in \{1, 2, \dots, dv\}$ is the order of the check node updated in an iteration and dv is the variable node degree. After an iteration is finished, the posterior reliability values after the last check update $\Lambda_n^{(q)}[dv]$ will be used as $\Lambda_n^{(q+1)}[0]$ in the following iteration. Equations (7)–(10) are derived by merging the variable node process and the soft-output updating process (2)–(3) with the CN update process (4)–(5). The variable node process is spread on the check node updating and the posterior reliability value $\Lambda_n^{(q+1)}$, is refreshed after every check node update. The key advantage of the layered schedule is that the immediate updates of the posterior messages are used and propagated to the next layers for its updating within the iteration [14], which increases the convergence speed and reduces the average number of iteration by up to 50% [9].

The computation of (6) and (8) are complicated and difficult for hardware implementation. Low complexity algorithms such as min-sum approximation have been proposed to reduce the computation complexity [24]. In the min-sum decoding algorithm, the computation of (8) is approximated and expressed by

$$\left|R_{m,n}^{(q+1)}\right| = \min_{j \in \{N_m \setminus n\}} \left|\Gamma_{m,j}^{(q+1)}\right|. \quad (11)$$

Thus for a check node m to compute the magnitudes of the outgoing messages, only two of the incoming messages with the smallest magnitudes have to be determined. The computation complexity of (8) is significantly reduced. Furthermore, the storage of the outgoing messages has been reduced to only two as opposed to dc , where dc stands for the check node degree (i.e., the number of the neighboring variable nodes of a check node), because $dc-1$ variable nodes share the same outgoing message [24]. In order to achieve better performance and maintain the similar computation complexity and storage requirement of the min-sum approximation, the variants of the min-sum, such as offset min-sum [16], [17] and two-output approximation [15], have also been proposed and adopted in the hardware design.

B. The Layered Decoding Architecture

The layered decoding algorithm has been adopted in many designs [9]–[18] due to their high convergence speed and easy adaptation to the flexible LDPC codes. In this section, the

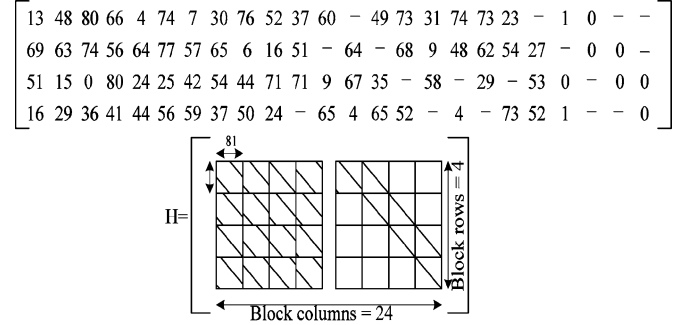


Fig. 2. Base matrix for a rate 5/6 with sub-block size of 81 LDPC code defined in IEEE 802.11n.

decoder architecture with layered decoding algorithm for the type of the architecture-aware LDPC codes (AA-LDPC) [9] is briefly introduced. Architecture-aware codes were proposed to facilitate the hardware design of the decoder. They are structured codes, whose parity-check matrix is built according to specific patterns. They are suitable for VLSI design, because the interconnection of the decoder is regular and simple and the trade-off between throughput and hardware complexity can be easily made. Since they support an efficient partial-parallel hardware VLSI implementation, AA-LDPC codes have been adopted in several modern communication standards, such as DVB-S2 [3], IEEE 802.16e [4] and IEEE 802.11n [5]. Fig. 2 shows an example of such parity-check matrix which is a LDPC code defined in IEEE 802.11n [5]. It is of rate 5/6 with sub-block size (i.e., the size of the identity sub-matrix) of 81. The parity-check matrix is composed of null sub-matrix or identity sub-matrix with different cyclic shifts. The numbers stand for the cyclic shift value of the identity sub-matrix, and the “–” stands for null sub-matrix.

Several VLSI architectures have been proposed for the decoder of these systems [13]–[19], and layered decoding algorithm is commonly adopted in the design. A block diagram of these decoders is shown in Fig. 3 [13]–[19]. In the decoder, multiple soft-in soft-out (SISO) units work in parallel to calculate multiple check node process for a layer. The Channel RAM is used to store the input LLR value of the received data initially. During the iteration of the decoding, it is used to store the posterior reliability values of the variable nodes. The shifter is used to perform the cyclic shift of the soft output messages so that the correct message is read out from the Channel RAM and sent to the corresponding SISO for calculation based on the base matrix. The Sub-array is used to perform the subtraction of (9), and the results will be sent to the SISO unit and the memory used to store these intermediate results (i.e., FIFO in Fig. 3), at the same time. The SISO unit performs the check node process of (7) and (8). The two-output approximation [15] is used for the SISO computation, and two outgoing magnitudes are generated for a check node. One is for the least reliable incoming variable node, and the other is for the rest of the variable nodes. Thus, the SISO unit, for every check node, will generate the signs for the outgoing messages of all the variable nodes, two magnitudes and an index [23]. The index is used to select the two magnitudes for the update process in the Add-array. The data generated by the SISO will be stored in the Message RAM. The Add-array

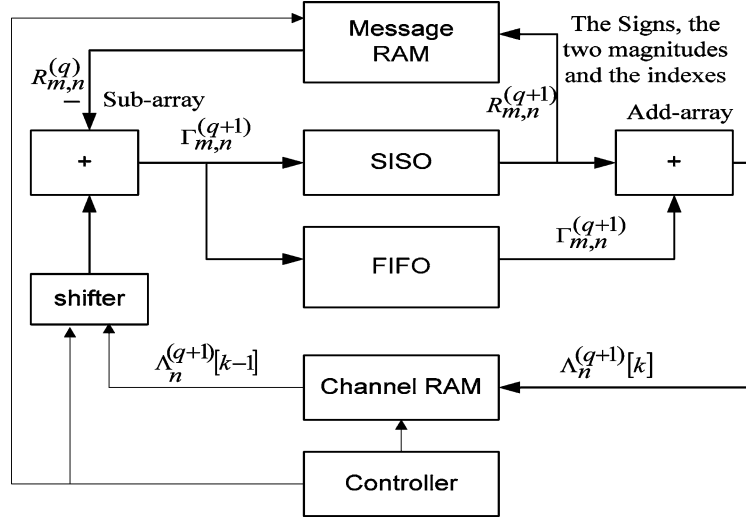


Fig. 3. Block diagram of the layered LDPC decoder.

performs the addition of (10), by taking the output of the SISO and the intermediate results stored in the FIFO. The results of the Add-array will be written back to the Channel RAM. To increase the throughput, pipeline operation of the decoder is adopted in the design [15], [16].

To reduce the power consumption of the Channel RAM for a layered decoding architecture, memory-bypassing scheme has been proposed in [10] by reducing the amount of memory access of the Channel RAM, which is briefly introduced in Section II-C.

C. The Memory Bypassing Scheme

In the layer decoding architecture, during the decoding, for every layer, the soft messages are read from and wrote into the Channel RAM and the FIFO every cycle. The Channel RAM stores the soft posterior reliability values of the variable nodes. The updated values are stored back from the Add-array and will be used in the decoding of the subsequent layer. In the memory-bypassing scheme [10], when two consecutive layers have non-null entry at the same column, the results of the Add-array can be directly sent to the cyclic shifter and used for the decoding of the next layer without the need of storing the intermediate result in the Channel RAM. The memory bypassing scheme saves the write operation for the current layer and the read operation for the next layer. Fig. 4 shows an example. Fig. 4(a) shows a base matrix with three layers and Fig. 4(b) shows the timing diagram of the pipeline. Without any memory bypassing, the number of read and write access of the Channel RAM is equal to the non-null entries in the matrix. In this example, the total number of read and write operation is 12. If by-passing scheme is employed, instead of writing back the channel RAM, the updated soft output values are used directly for the decoding of the next layer, the number of memory access is reduced. For example memory access for columns 0 and 2 can be bypassed when the decoding proceeds from layer 0 to layer 1; memory access for columns 0 and 1 can be bypassed for the second layer decoding and memory access for columns 0 and 3 can be bypassed for the third layer decoding. Thus, 6 out of

the 12 read and write operations are eliminated, and 50% of the power consumption of the Channel RAM can be saved.

Assume the pipelined architecture takes two clock cycles for the cyclic shifter, Sub-array, the SISO and the Add-array to finish the computation after the last incoming variable node is read in, a detail timing diagram showing the operation of the decoder is shown in Fig. 4(c). The order of read and write of the Channel RAM is following the natural order stated in the base matrix. Fig. 4(d) shows the memory bypassing scheme introduced in [10]. We denote it as simple memory bypassing. Here column 0 and 2 are written earlier for layer 0 and columns 0 and 2 are scheduled later for layer 1 so that overlap can be achieved. There exist some column orders that can result in maximum overlap between two layers. However, for the subsequent layers the memory overlap may not be optimized. Due to data dependency, the memory write of a certain column for the existing layer should finish before or at the same time with the reading of the same column for the subsequent layer. In order to achieve that, the decoding of the third layer has to be delayed to align the memory access and idling cycles are inserted in the decoding pipeline. We need to add idling cycle in order to maximize the overlap and even that there is still one potential overlap (W3, R3) in the third layer that cannot be achieved. From that we can see that the simple memory bypassing scheme cannot realize all the potential memory bypass operation available for the decoding matrix or extra idling cycles are needed to be inserted. In Section III, we will present an improved memory bypassing scheme which can achieve memory bypass for all the overlapped columns.

III. AN IMPROVED MEMORY BYPASSING SCHEME

In order to achieve the optimal amount of memory bypassing, we de-couple the read and write order of the Channel RAM and the memory storing the intermediate messages (i.e., FIFO in the traditional design) for a layer. This is shown in Fig. 5(a). We can see that all the potential bypassings are achieved and at the same time the idling cycles are minimized. As the messages read out from the Channel RAM will be stored in the intermediate data RAM (i.e., FIFO in the traditional design) after the subtraction

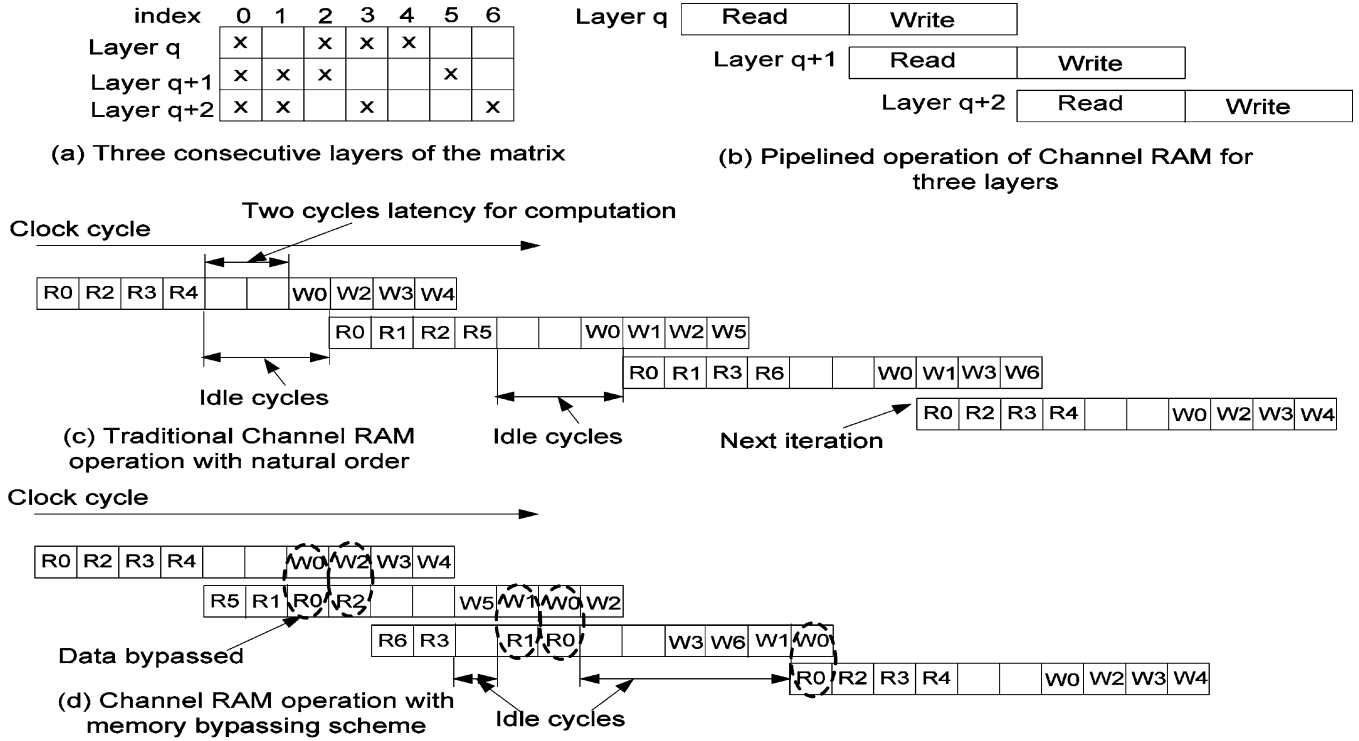


Fig. 4. The memory bypassing operation for the Channel RAM in the layered LDPC decoder.

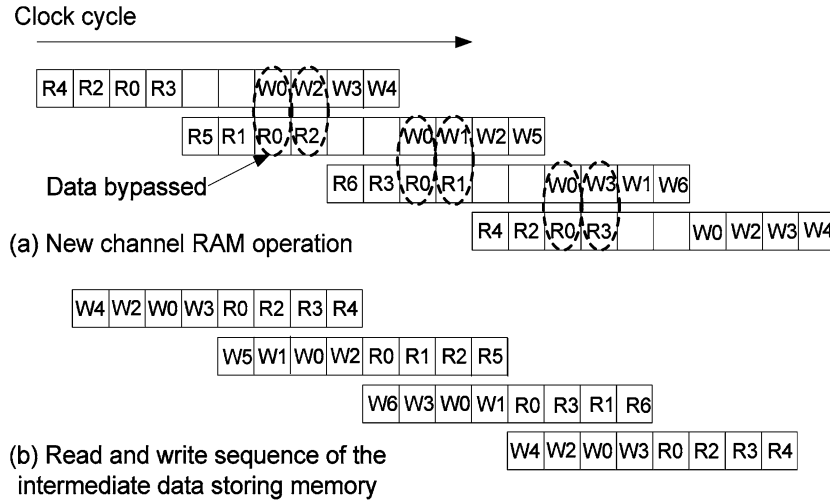


Fig. 5. Memory operations with different read and write order for the matrix shown in Fig. 4.

and will be read out from the intermediate data RAM for the check node update, the read order of the Channel RAM is the write order of the intermediate data RAM and the read order of the intermediate data RAM is the write order of the Channel RAM. The access sequence of the intermediate data RAM is shown in Fig. 5(b).

In the above example, we limited our discussion to the following assumptions.

- 1) Only overlapping between two consecutive layers are considered for the memory bypassing.
- 2) The number of the latency cycles is 2, i.e., it takes two clock cycles for the cyclic shifter, Sub-array, the SISO and the Add-array to finish the computation after the last incoming variable node is read in.

To optimize the number of memory bypass, we can consider the overlapping of more layers. For example, in Fig. 4(b), the first layer and the third layer have non-null entry at column 3, and this overlapping can be used for memory bypassing. The memory operations considering the overlapping of the three consecutive layers are shown in Fig. 6.

For this example, if we also consider the overlapping of the first and the third layer, two more memory access can be bypassed (i.e., the write operation W_3 in the first layer and W_2 in the second layer can be bypassed with the read operation R_3 in the third layer and R_2 in the first layer of the next decoding iteration.) Considering the overlapping of three consecutive layers, the memory-bypassing operation can be divided into two cases: memory-bypassing between layers $q+2$ and

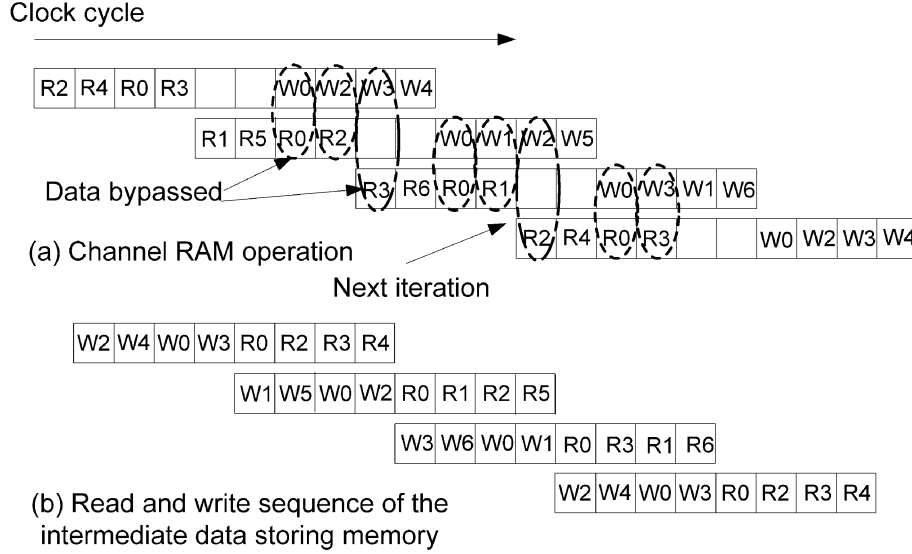


Fig. 6. Memory operations by considering the overlapping of three consecutive layers for the matrix shown in Fig. 4.

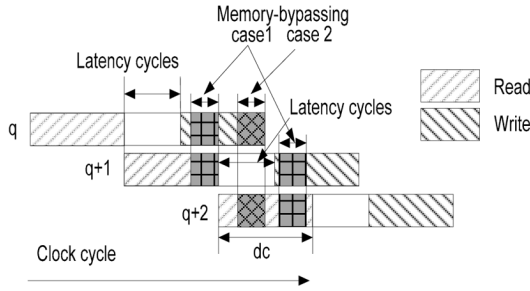


Fig. 7. Channel RAM operation when considering the overlapping among three consecutive layers.

$q + 1$, memory-bypassing between layers $q + 2$ and q . The potential number of memory-bypassing that can be achieved for layer $q + 2$ is the number of the non-null entry that the layer $q + 2$ are in common with the above two layers. However, this amount of the memory-bypassing may not always be achieved. The amount of memory -bypassing between layer $q + 2$ and layer q (i.e., case2 shown in Fig. 7) is limited by the latency cycles. When the latency cycles are smaller than the number of the non-null entry that the current layer $q+2$ are in common with the layers q but not in common with layer $q+1$, some of the memory operation for the overlapped columns cannot be bypassed. In this case, to increase the amount of memory-bypassing operation, the latency clock cycles have to be increased. We limit the memory-bypassing to three consecutive layers. Since the check node degree is in general larger than the latency cycles, the write operation of layer q will finish earlier than the read operation of the layer $q+3$ unless we insert many latency clock cycles. Thus, the memory-bypassing of four or more layers is not considered.

The amount of the overlapped columns indicates the number of memory-bypassing operation that we can achieve. The amount of the overlap directly depends on the order of the layer decoding. To achieve the optimal amount of memory bypassing, the optimal decoding order has to be determined, and also the non-null entries in every layer has to be properly scheduled such that the memory bypassing can be applied for every overlapped column. In Section IV, the algorithm for finding the

TABLE I
TOTAL NUMBER OF THE OVERLAPPED COLUMNS WHEN CONSIDERING THE OVERLAPPING OF TWO CONSECUTIVE LAYERS FOR THE IEEE 802.11N LDPC CODES

Rate \ Order	1/2	2/3	3/4	5/6
Best case order	49	55	59	65
Natural order	48	50	59	65
Worst case order	30	42	49	63

TABLE II
TOTAL NUMBER OF THE OVERLAPPED COLUMNS WHEN CONSIDERING THE OVERLAPPING OF THREE CONSECUTIVE LAYERS FOR THE IEEE 802.11N LDPC CODES

Rate \ Order	1/2	2/3	3/4	5/6
Best case order	61	67	70	78
Natural order	53	65	67	76
Time required (second)	68550.28	4.02	0.19	0.04

optimal decoding order and the scheduling algorithm for the non-null entries inside each layer will be presented.

IV. THE QUICK SEARCHING ALGORITHM

To obtain the optimum decoding order, we can use brute-force method to list out all the permutation orders of the layers and then compute the number of overlapped columns of each permutation. A few experiments have been done for the example codes defined in IEEE 802.11n. The results are shown in Tables I and II which also shows the time required for the calculation. Table I shows the results when we only consider the overlapping of two consecutive layers. We also compare the number of overlapped column obtained using different decoding orders, namely the best, the natural and the worst order. Table II shows the corresponding results when we consider the overlapping of three consecutive layers. From the results we can see that the total number of overlapped columns when considering the overlapping of three consecutive layers using the optimal order is increased by 12.9%–19.3% and the memory access of the Channel RAM is reduced by the same amount.

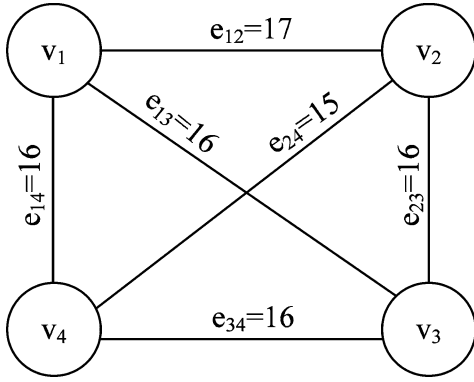


Fig. 8. Undirected graph $G = (V, E)$ for the rate 5/6 code LDPC code shown in Fig. 2.

Table II shows that for the codes with small number of the layers, the brute-force method is still practical. However, when the base matrix becomes larger, the time required for the calculation will increase dramatically. It is infeasible to use the brute-force method to find the best order of the layers for the base matrix with large number of layers. For example, the LDPC codes defined in DVB-S2 have 180 layers! The complexity of the brute-force method is $n!$ and will quickly become impractical when the number n increases.

In order to reduce the computation time for the searching algorithm, the problem of finding the optimal decoding order of the layers which has the maximum amount of overlapping is analyzed. A quick searching algorithm is proposed and the results for a few LDPC codes are presented. For better illustration, the algorithm finding the best order of the layers having the maximum amount of overlapped columns between two consecutive layers is considered first. We denote it as two-layer overlapping. It can be formulated as a graph problem. Let $G = (V, E)$ be a complete graph where V and E are the set of the nodes and edges of the graph, respectively. A node v_i represents row i of the base matrix and the cost of edge $e_{i,j}$ is the number of overlap between rows i and j . The problem of finding the optimal orders is the same as that finding the path starting from any of the node in G , visiting all the other nodes exactly once and returning back to the starting node and that summation of costs of the edges of the path is maximum. Fig. 8 shows the graph representing the rate 5/6 code of IEEE 802.11n. The problem is equivalent to the well known traveling salesman problem (TSP) which is known to be NP-hard. For problems that have small number of nodes, such as the one shown in Fig. 8, brute force approach can be used to find the optimal solution. For problem with large size, heuristic algorithm is needed to generate a good solution.

The problem of finding the optimal order of the layers that considers overlapping among more than two consecutive layers is similar to the two layer overlapping problem except that the calculation of the cost function is more complicated. Hence, the computation complexity is also NP-hard and heuristic algorithm is required to find a good solution for problem with large value of n .

In this work, we used simulated annealing [28] to implement the heuristic algorithm for the two-layer-overlapping and three-layer-overlapping problems. The cost function used in the

TABLE III
TOTAL NUMBER OF THE OVERLAPPED COLUMNS FOR THE IEEE 802.16e LDPC CODES

802.16e Sub-block size 96	Two layer		Three layer	
	Natural order	Solution found by simulated annealing	Natural order	Solution found by simulated annealing
1/2	23	38	35	50
2/3 A	35	42	55	59
2/3 B	9	52	73	73
3/4 A	51	51	70	75
3/4 B	55	62	75	75
5/6	67	67	77	79

TABLE IV
TOTAL NUMBER OF THE OVERLAPPED COLUMNS FOR THE IEEE DVB-S2 LDPC CODES

DVB-S2 Sub-block size 360	Two layer		Three layer	
	Natural order	Solution found by simulated annealing	Natural order	Solution found by simulated annealing
1/4	9	146	25	176
1/3	14	157	48	196
2/5	26	170	55	221
1/2	24	139	42	182
3/5	70	185	123	265
2/3	34	131	71	183
3/4	47	136	106	197
4/5	71	130	113	194
5/6	90	148	146	224
8/9	54	86	106	145
9/10	55	97	113	159

simulated annealing is the number of the overlapped columns for the base-matrix for a particular order of the layers. For each move, a pair of layers is randomly selected and their orders in the decoding sequence are swapped. The cost function is updated for the new sequence. The new sequence is accepted or rejected accordingly to the annealing schedule. Fig. 9 shows the detail flow diagram of the simulated annealing algorithm used in this work.

We applied the algorithm on different LDPC codes, including codes for IEEE 802.11n, IEEE 802.16e, and DVB-S2. The experimental results are summarized in Tables III–IV. For small LDPC codes such as the codes for IEEE 802.11n and IEEE 802.16e, the simulated annealing algorithm always converges to the optimal solution. For large LDPC codes, such as the codes used in DVB-S2, the simulated annealing gives solutions that result in significant increases in overlapping. Table III shows that for the codes used in IEEE 802.16e [4], 65.8%–98.8% of the access of the posterior reliability values in the Channel RAM can be bypassed. Table IV shows that for the codes used in DVB-S2 [3], 30.9%–65.9% of the access of the posterior reliability value in the Channel RAM can be bypassed. It can be seen that considering overlapping among more layers and using optimum decoding order have a significant effect on the increase in the number of memory bypassing.

In order to implement the memory bypassing scheme, after determining the optimal decoding order of the layers, the order

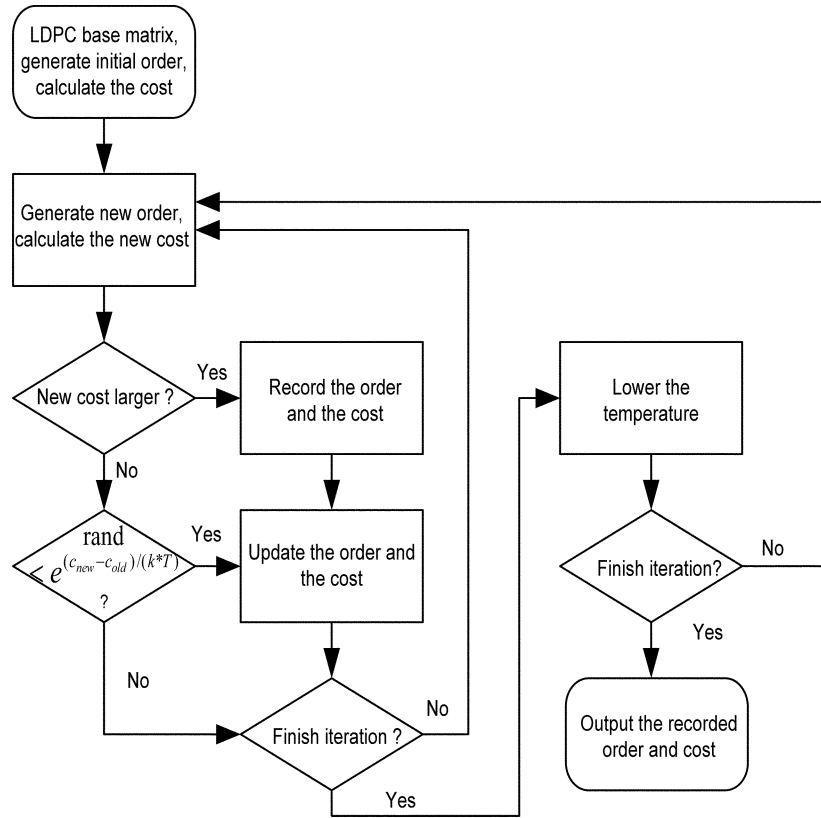


Fig. 9. The flow of the simulated annealing algorithm for finding the order of the layers.

of the column decoding of each layer has to be properly scheduled. Since the read and write order of the Channel RAM for a layer are decoupled, they can be scheduled differently and independently. As shown in Fig. 7, for the write order of layer q , the non-null entries that the current layer q are in common with the next layer $q + 1$ are written first. The non-null entries that the layer $q + 2$ are in common with the layer q but not with the layer $q + 1$ are written at the end. The other non-null entries are then scheduled in between randomly. From Fig. 7, we can also see that if the latency of the decoding data-path is zero, i.e., the SISO can start writing to the Channel RAM for a certain layer, e.g., layer $q + 1$, immediately after all the inputs are read from the Channel RAM, then the memory bypassing between layers q and $q + 2$ cannot be done. It also means that if the number of overlapping between layers q and $q + 2$ is larger than the latency of the data-path, some of the potential memory bypassing cannot be realized. In this case, in order to achieve the maximum memory bypassing operation, we need to add idle clock cycles in the pipeline. When the latency is larger than the number of overlapping between layers q and $q + 2$, memory bypassing can always be achieved for all the overlapped columns without the need of inserting idle clock cycles. In Section V, the architecture of the LDPC decoder implementing the improved memory-bypassing scheme will be presented.

V. LDPC DECODER ARCHITECTURE IMPLEMENTING THE MEMORY BYPASSING SCHEME

The block diagram of the proposed LDPC decoder for IEEE 802.11n is shown in Fig. 10. Since the sub-block size is 81,

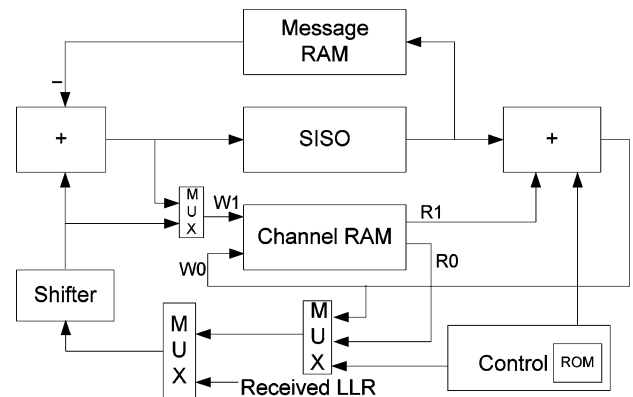


Fig. 10. Block diagram of the proposed LDPC decoder.

we use 81 soft-in soft-out (SISO) units working in parallel to calculate multiple check nodes processing for a layer at the same time. The operation of the shifter, the sub-array and the SISO is the same as the traditional layered decoding architectures [13]–[15]. The received messages are quantized into 5 bit and the bit-width of the soft output for the variable nodes is 6. The two-output approximation approach [15] is used to implement the SISO. For every check node, two magnitudes, an index, and signs from all variable nodes are generated to represent the check node message and stored in the message RAM [23]. The message RAM is composed of one 88×88 -bit dual port SRAM and 14 24×45 -bit dual port SRAMs. The one 88×81 -bit dual

TABLE V
COMPARISON CLOCK CYCLES REQUIRED PER ITERATION AND IDLE CYCLES IN PERCENTAGE

Rate		1/2	2/3	3/4	5/6
Cycle per iteration	Traditional design	158	135	121	102
	Memory bypassing	90	88	86	80
	Simple bypassing	133	120	107	91
	[18]	94	89	90	95
Idle cycle (%)	Traditional	45.7	34.8	29.8	22.5
	Memory bypassing	4.4	0.0	1.2	1.3
	Simple bypassing	35.3	26.7	20.6	13.2
	[18]	7.5	1.0	4.9	14.5

port SRAM is used to store the signs of the check node messages, and the 14 24×45 -bit dual port SRAMs are used to store the magnitudes and the indexes which represent the magnitudes of the check node messages. The decoder will stop decoding if the signs of the messages $\Lambda_n^{(q+1)}$ in one iteration during the decoding satisfy all the parity checks or the number of iteration equals to a pre-defined value (In our design, the value is equal to 15). In order to minimize the memory access of the Channel RAM, the decoding order of the layers is determined by the algorithm introduced in the previous section. After determining the decoding order of the layers, the read and write order of the Channel RAM for the non-zero entries within a layer is scheduled to achieve the memory bypassing for all the overlapped columns and minimize the idle cycles due to the data dependency of the layers. The read and write order of the intermediate data RAM is then fixed, as they are the same as the write and read order of the Channel RAM, respectively.

In the traditional decoder, the FIFO and the Channel RAM are implemented separately, because of the addressing of the FIFO and Channel RAM are different. By de-coupling the read and write order of the intermediate data RAM, the intermediate data RAM and the Channel RAM can be combined because the messages stored in the same address in the Channel RAM $\Lambda_n^{(q+1)}$ and the intermediate data RAM $\Gamma_{m,n}^{(q+1)}$ are not needed at the same time. When the intermediate data RAM is storing the message $\Gamma_{m,n}^{(q+1)}$ at the location n , the message stored at the location n of the Channel RAM $\Lambda_n^{(q+1)}$ is not needed, and vice versa. Instead of using two separate memories, we use a single four port memory (two read ports and two write ports), to implement both the Channel RAM and the intermediate data RAM. By doing so, the area of the required memory is reduced. We denote the combined memory as the new Channel RAM. The new Channel RAM is composed of $6 \times 24 \times 81$ bit four port SRAMs and stores the input LLR values of the received data initially. Each entry of the new Channel RAM is dedicated to store the messages of the 81 variable nodes in the base-matrix. One pair of the read and write ports (R0 and W0) are used for the read and write access of the original Channel RAM and another pair of the read and write ports (R1 and W1) are used for the read and write access of the original intermediate data RAM. If the updated results will be used directly in the decoding of the following layers, they will be sent to the shifter directly through a mux-array. The write port W0 and the read port R0 are disabled. Otherwise, the updated messages will be written into the new Channel RAM through the write port W0 and the messages needed in the next layer decoding are read out through the read port R0.

A bank of muxes is added to select the output of the Add-array and that of the Channel RAM and pipeline registers are added after the Add-array, to implement the memory bypassing scheme. Because the order of the messages entering the SISO (i.e., the read order of the read port R0) and the order of the messages updated in the Add-array (i.e., the read order of the read port R1) are different, the index generated in the SISO indicating the position of the least reliable incoming messages will be incorrect for the update process. A ROM containing the order of the updated process (i.e., read order of the read port R1) is added and it is used together with the index generated in the SISO to select the two magnitudes for the update process. The additional hardware is easy to implement and the overhead in area and power is very small.

By de-coupling the read and write order of the Channel RAM and using the memory bypassing scheme, the number of read and write access of the Channel RAM is reduced by 70.9%–98.7% depending on the codes. At the same time, the idle cycles due to the data dependency of messages are minimized.

VI. EXPERIMENTAL RESULTS

We implemented the proposed LDPC decoder to demonstrate the performance of the improved memory by-passing scheme. We also implemented the traditional layered decoding architecture [14], [15] and the LDPC decoder using simple memory bypassing scheme for comparison. For all the designs, the bit-width for the soft output messages is set to 6. The decoders were implemented and synthesized with Synopsys Design Compiler using the Artisan's TSMC 0.18 μm standard cell library. The power consumption of the embedded SRAM is characterized by hspice simulation with the TSMC 0.18 μm process. The power consumption of the decoder was simulated using Synopsys VCS-MX and Prime-Time. The supply voltage is 1.8V and the clock frequency is 250MHz.

The first experiment compares the clock cycle required, the idle cycles and hence the throughput of the decoder. We also compared with the design in [18], which uses the reversal order of the read operation as the order of the write operation to reduce the idle cycles. Table V summaries the results. The required numbers of idle cycles reported in [18] are also included in Table V. Compared with the traditional decoder using the natural order (i.e., the order specified in the standard), the proposed decoder with the memory bypassing scheme decouples the read and write order of the memory and hence can reduce the number of idle cycles by 21.2%–41.3%. Compared with the simple memory bypassing scheme of which the read and write

TABLE VI
ENERGY CONSUMPTION (PJ/Bit/Iteration) OF THE THREE LDPC DECODERS WHEN OPERATED IN 250 MHz

Rate		1/2	2/3	3/4	5/6
Message RAM	Traditional design	16.5	13.1	10.8	8.7
	Simple bypassing	16.5	13.1	10.8	8.7
	Our design	16.5	13.1	10.8	8.7
Logic units	Traditional design	62.3	54.6	49.4	43.9
	Simple bypassing	60.9	53.0	48.7	43.9
	Our design	57.7	50.0	47.5	43.8
Channel RAM + FIFO	Traditional design	64.7	66.6	65.5	62.4
	Simple bypassing	47.1	48.3	43.8	38.6
	Our design	40.4	40.2	38.4	32.7
Total	Traditional design	143.5	134.3	125.7	115.0
	Simple bypassing	124.5	114.4	103.3	91.2
	Our design	114.6	103.3	96.8	85.2

TABLE VII
COMPARISON THE DIFFERENT LDPC DECODER IMPLEMENTATIONS

	Memory bypassing LDPC decoder	Traditional LDPC decoder	IEEE 802.11n LDPC decoder [18]	IEEE 802.16e LDPC decoder [20]	TDMP LDPC Decoder [12]
Throughput	503 Mb/s (250MHz, 10 iter, synthesis result)	294.5Mb/s (250MHz, 10 iter., synthesis result)	356Mb/s (240MHz, 13~14 iter., synthesis result)	105Mb/s (150MHz, 20 iter.)	640Mb/s (125MHz, 10 iter.)
Area	2.67 mm ² (1.26 mm ² logic part)	2.67 mm ² (1.24 mm ² logic part)	0.74 mm ² (0.45 mm ² logic part)	6.25 mm ²	14.3 mm ²
Power	463 mW	339.5 mW	234.6 mW	264 mW	787 mW
Energy efficiency	92.05 (pJ/bit/iter.)	115.26 (pJ/bit/iter.)	—	125 (pJ/bit/iter.)	123 (pJ/bit/iter.)
CMOS technology	180 nm, 1.8V	180 nm, 1.8V	65 nm, 1.2V	90nm, 1.0V	180nm, 1.8V

order of the Channel RAM are not decoupled, the number of idle cycle is reduced by 11.9%–30.6%. Compared with the design in [18], the number of idle cycles is reduced by 1.0%–13.2%. The idle clock cycle in the decoder using the proposed memory bypassing scheme is only due to the irregular check node degrees. The idle clock cycles due to the data dependency, i.e., the updated message is computed before it can be used in another layer [21], [22] are all eliminated.

The next experiment compares the power consumption of different decoders. Because the clock cycles required per iteration for the decoders are different, we used the energy efficiency of the decoders for comparison instead. The average energy consumptions for decoding a block of data for different code rate modes are shown in Table VI. It can be seen that the decoder using the proposed memory bypassing scheme reduces the energy consumption of the channel RAM and FIFO by 37.6%–47.5%, comparing with the traditional decoder. The corresponding reduction in energy is 27.2% to 38.1% when comparing with the traditional architecture using simple bypassing scheme. The overall energy reduction of the decoder is reduced by 20.1%–25.9% and 13.2%–20.7%. The reduction will be more for other LDPC codes such as DVB-S2 and IEEE 802.16e as the amount of memory bypassing that can

be achieved by using the improved memory bypassing scheme is increased significantly as shown in Tables III and IV.

Finally we compare the overall power consumption of the proposed decoder with other LDPC decoder implementations published in the literature. The comparisons are summarized in Table VII. Although it is hard to directly compare the different architectures since the LDPC codes are different and the implementation technologies are also different, the comparison gives some idea on the energy efficiency of different designs.

VII. CONCLUSION

We presented an improved memory-bypassing scheme to reduce the memory access and hence the energy consumption of the LDPC decoder by exploiting the characteristic of the LDPC parity check matrix. Searching algorithm to find the optimal decoding order that results in maximum number of memory bypassing was proposed. The corresponding architecture supporting the proposed scheme was also presented. Experimental results show that optimum reduction in the memory access can be achieved for LDPC decoder targeting IEEE 802.11n specification and the memory access is reduced by 12.9%–19.3%. In addition, the idling cycle is reduced by 1.0%–13.2%, compared with the state-of-the-art design.

REFERENCES

- [1] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. IT-8, pp. 21–28, Jan. 1962.
- [2] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, Mar. 1999.
- [3] Digital Video Broadcasting (DVB); Second Generation Framing Structure, Channel Coding and Modulation Systems for Broadcasting, Interactive Services, News Gathering and Other Broadband Satellite Applications 2004.
- [4] *LDPC coding for OFDMA PHY. 802.16REVe Sponsor Ballot Recirculation Comment* 2004, IEEE C802.16e-04/141r2.
- [5] *Joint Proposal: High Throughput Extension to the 802.11 Standard: PHY. IEEE P802.11 Wireless LANs* 2006, IEEE 802.11-05/1102r4.
- [6] R. Tanner, "A recursive approach to low complexity codes," *IEEE Trans. Inf. Theory*, vol. 27, no. 5, pp. 533–547, Sep. 1981.
- [7] A. J. Blanksby and C. J. Howland, "A 690-mW 1-Gb/s 1024-b, rate 1/2 low-density parity-check code decoder," *IEEE J. Solid-State Circuits*, vol. 37, pp. 404–412, Mar. 2002.
- [8] J. Zhang and M. P. C. Fossorier, "Shuffled iterative decoding," *IEEE Trans. Commun.*, vol. 53, no. 2, pp. 209–213, Feb. 2005.
- [9] M. M. Mansour and N. R. Shanbhag, "High-throughput LDPC decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 11, no. 6, pp. 976–996, Dec. 2003.
- [10] E. Cavus, "Techniques for the decoding of low density parity check codes: efficient simulation, algorithm improvement and implementation," Ph.D. dissertation, UCLA, Los Angeles, 2007.
- [11] M. Rovini, F. Rossi, P. Cio, N. L'Insalata, and L. Fanucci, "Layered decoding of non-layered LDPC codes," in *Proc. Euromicro Conf. Digital Syst. Design*, Aug.-Sep. 2006, pp. 537–544.
- [12] M. Mansour and N. R. Shanbhag, "A 640-Mb/s 2048-bit programmable LDPC decoder chip," *IEEE J. Solid-State Circuits*, vol. 41, pp. 684–698, Mar. 2006.
- [13] J. Dielissen, A. Hekstra, and V. Berg, "Low cost LDPC decoder for DVB-S2," presented at the Design, Automation Test in Eur., Munich, Germany, Mar. 2006.
- [14] T. Brack, M. Alles, T. Lehnigk-Emden, F. Kienle, N. Wehn, N. E. L'Insalata, F. Rossi, M. Rovini, and L. Fanucci, "Low complexity LDPC code decoders for next generation standards," in *Proc. Design, Automation and Test in Europe*, Apr. 2007, pp. 1–6.
- [15] G. Gentile, M. Rovini, and L. Fanucci, "Low-complexity architectures of a decoder for IEEE 802.16e LDPC codes," in *Proc. Euromicro Conf. Digital Syst. Design Architectures, Methods Tools*, Aug. 2007, pp. 369–375.
- [16] Y. Sun, M. Karkooti, and J. R. Cavallaro, "VLSI decoder architecture for high throughput, variable block-Size and multi-rate LDPC codes," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2007, pp. 2104–2107.
- [17] K. K. Gunnam, G. Choi, W. Wang, and M. B. Yeary, "Multi-rate layered decoder architecture for block LDPC codes of the IEEE 802.11n wireless standard," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2007, pp. 1645–1648.
- [18] M. Rovini, G. Gentile, F. Rossi, and L. Fanucci, "A scalable decoder architecture for IEEE 802.11n LDPC codes," in *Proc. Global Telecommun. Conf.*, Nov. 2007, pp. 3270–3274.
- [19] X. Y. Shih, C. Z. Zhan, C. H. Lin, and A. Y. Wu, "An 8.29 mm² 52 mW multi-mode LDPC decoder design for mobile wimax system in 0.13 μ m CMOS process," *IEEE J. Solid-State Circuits*, vol. 43, no. 3, pp. 672–683, Mar. 2008.
- [20] C. H. Liu, S. W. Yen, C. L. Chen, H. C. Chang, C. Y. Lee, Y. S. Hsu, and S. J. Jou, "An LDPC decoder chip based on self-routing network for IEEE 802.16e applications," *IEEE J. Solid-State Circuits*, vol. 43, no. 3, pp. 684–694, Mar. 2008.
- [21] T. Kuo and A. N. Willson, "Enhanced delta-based layered decoding of WiMAX QC-LDPC codes," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2008, pp. 524–527.
- [22] T. Brack, M. Alles, T. Lehnigk-Emden, F. Kienle, N. Wehn, F. Berens, and A. Ruegg, "A survey on LDPC codes and decoders for OFDM-based UWB systems," in *Proc. IEEE 65th Veh. Technol. Conf.*, Apr. 2007, pp. 1549–1553.
- [23] Z. Wang and Z. Cui, "A memory efficient partially parallel decoder architecture for QC-LDPC codes," in *Proc. 39th Asilomar Conf. Signals, Systems, Comput.*, Oct. 2005, pp. 729–733.
- [24] J. Chen, A. Dholakia, E. Eleftheriou, M. P. C. Fossorier, and X.-Y. Hu, "Reduced-complexity decoding of LDPC codes," *IEEE Trans. Commun.*, vol. 53, no. 8, pp. 1288–1299, Aug. 2005.
- [25] F. Zarkeshvari and A. Banihashemi, "On implementation of min-sum algorithm for decoding low-density parity-check (LDPC) codes," in *Proc. IEEE GLOBECOM*, Nov. 2002, vol. 2, pp. 1349–1353.
- [26] A. Darabiha, A. C. Carusone, and F. R. Kschischang, "Clock-interlaced LDPC decoders with reduced interconnect complexity," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 55, no. 1, pp. 74–78, Jan. 2008.
- [27] A. Darabiha, A. C. Carusone, and F. R. Kschischang, "Power reduction techniques for LDPC decoders," *IEEE J. Solid-State Circuits*, vol. 43, no. 8, pp. 1835–1845, Aug. 2008.
- [28] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.



Jie Jin received the B.S. degree in electronic engineering from Xi'an Jiaotong University in 2003. He is currently working toward the Ph.D. degree in Electronic and Computer Engineering at the Hong Kong University of Science and Technology.

His research interests include VLSI architectures for low power communications and channel coding applications, and digital signal processing systems. He is currently working on VLSI architectures for advanced channel coding schemes such as low-density parity-check codes.



Chi-ying Tsui received the B.S. degree in electrical engineering from the University of Hong Kong and the Ph.D. degree in computer engineering from the University of Southern California, Los Angeles, in 1994.

He joined the Department of Electrical and Electronic Engineering, Hong Kong University of Science and Technology in 1994 and is currently an Associate Professor in the department. His research interests cover both design automation and design.

His current research includes designing VLSI architectures for low power multimedia and wireless applications, developing power management circuits and techniques for embedded portable devices and ultra-low power systems. He has published more than 140 referred technical journal and conference papers.

Dr. Tsui received the Best Paper Awards from the IEEE Transactions on VLSI Systems in 1995, IEEE/ACM ISLPED in 2007, and supervised the Best Student Paper Award of the 1999 IEEE ISCAS. He also received the Outstanding Design Awards of the IEEE ASP-DAC University Design Contest in 2004 and 2006.